

A Systematic Approach to Building Agentic Systems (Technical Case Study)

From Concept to MVP in under 3 hours!

TL;DR

- **The Build:** A real-time, agentic predictive queue rerouting system to solve physical stadium congestion.
- **The Constraint:** Core logic and MVP were architected in 3 hours during a live sprint. Full production deployment (security, UI theming, and cloud hosting) completed in 7 hours.
- **The Engine:** FastAPI (Backend), SQLite + Synthetic Data Simulator (Virtual Hardware), Streamlit (Frontend), and Gemini 2.5-Flash (Logic/Reasoning).

Table of Contents

The Agentic Shift	2
The Problem Statement	3
System Overview & Core Architecture	3
The Component Layers	4
The Technical Blueprint - Inside Stadium Flow	5
The Reasoning Engine (The Logic Loop)	5
The Hallucination Shield	7
Production Guardrails & Systemic Fixes	8
The Reality of the Build Timeline	9
Architectural Laws & Core Principles	10
The Orchestrator’s Blueprint	11
Appendix: References & Learning Tracks	14

The Shift to AI-Driven Orchestration

The way we build software has completely changed.

In 2026, the shift is real: we can now harness advanced AI models to orchestrate complete applications from start to finish.

Today, the only barrier between your idea and a working MVP is a **Well-Defined Prompt**.

You simply write the high-level instructions - a detailed **System Prompt** that defines:

- What to build and how it should behave?
- What exact tech stack would you like to use?
- What are the operational constraints and expected outcomes of the system?
- Who is the end user?
- What problem are you trying to solve, and what is your specific approach?
- How do you strategically plan to implement it?

Get these instructions right, and within *20 to 40 minutes*, you have a working Minimum Viable Product (MVP) running the core execution of your idea.

What is a **System Prompt**?

Think of a System Prompt as the **blueprint** you hand to a **junior developer**. It is a set of **guidelines** provided to an AI before it begins a task, defining its *persona*, *context*, and *operational boundaries*. It acts as the **structural framework** that shapes exactly how the AI *behaves*.

In the era of **AI-Driven Orchestration**, the System Prompt is your actual **source code**.

The AI handles the syntax and types the code, but you must architect the logic, the security, and the execution flow.

AI cannot read your mind (yet). You have to define exactly what to build and how to build it.

The Problem Statement:

"Design a solution that improves the physical event experience for attendees at large-scale sporting venues. The system should address challenges such as crowd movement, waiting times, and real-time coordination, while ensuring a seamless and enjoyable experience."

To ensure I didn't waste valuable development time on a loosely defined problem, I began by reverse-engineering the core operational friction of modern sports stadiums: **crowd congestion**.

I used AI to brainstorm potential concepts.

The goal wasn't just to find a cool idea, but to find a high-impact one that could actually be built and tested within a 3-5 hour window.

After generating a few concepts, I picked the one I could work on the best:

Predictive Queue Rerouting:

An agent that watches restroom or food stall wait times and dynamically re-routes guests. (For Example: 'Restroom A is full, but Restroom B is 30s away and empty.')

The idea was **simple, high-impact** and **executable**. That was all I needed for the day.

System Overview & Core Architecture:

Project Name: **Stadium Flow**

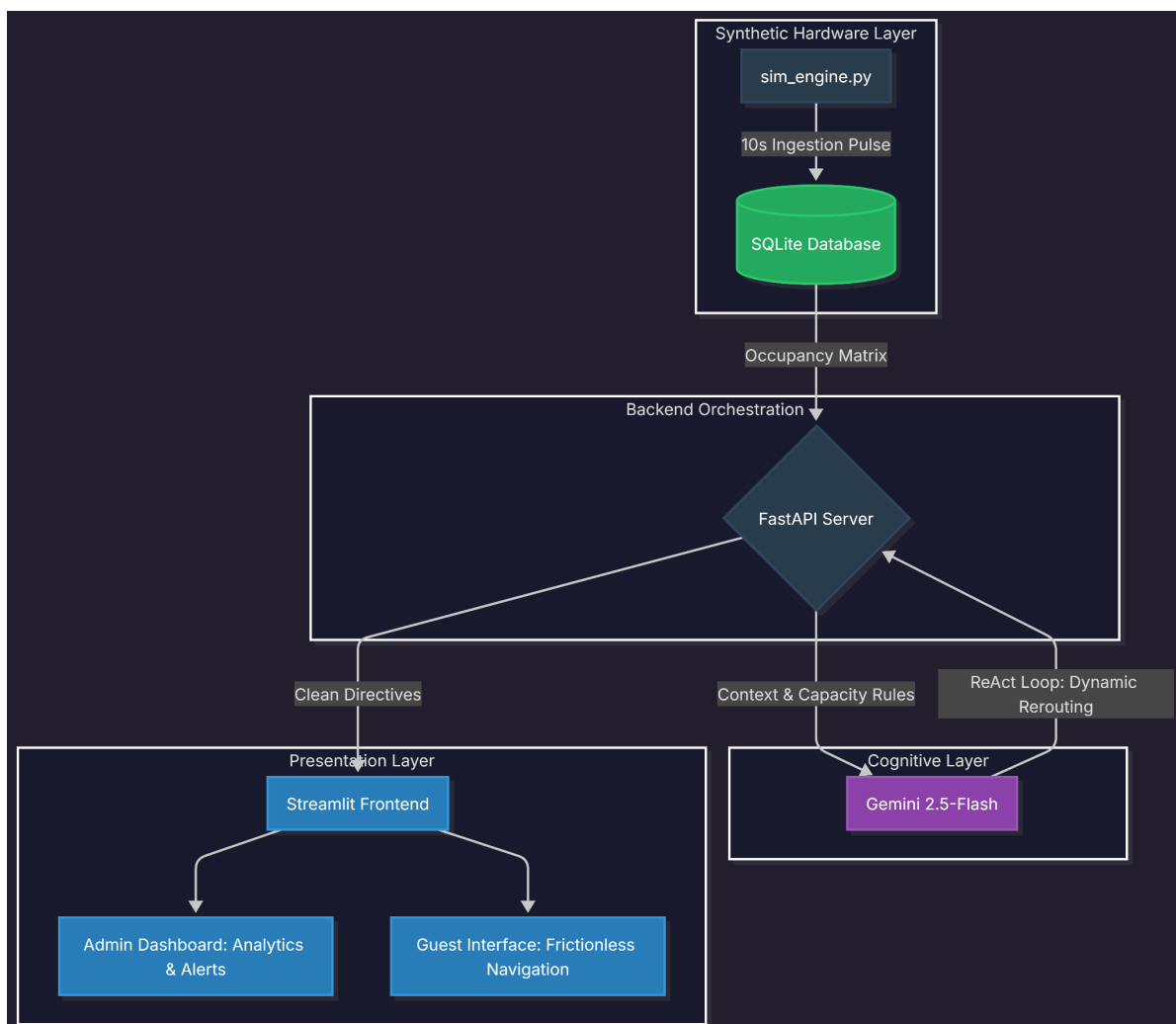
Description: A real-time, agentic predictive queue rerouting system designed to optimise crowd movement.

The Component Layers:

- **The Backend (FastAPI) - The HEART:**
 - Manage the `SQLite` database for stadium zones.
 - Since real-time sensor data was unavailable, the system runs a **concurrent simulation engine**.

- It generates synthetic data streams every **10 seconds** to simulate crowd movement.
- *The Impact:* This makes the system completely **hardware-agnostic**.
- To scale this in the real world, we just swap the simulated data source for a live IoT sensor feed, keeping the entire core system intact.
- **The Logic (Gemini 2.5-Flash) - The BRAIN:**
 - The **Reasoning Engine**.
 - It constantly evaluates the current zone occupancy against maximum capacity thresholds and generates human-readable recommendations (*exact breakdown of how this works in the next section*)
- **The Frontend (Streamlit) - The CANVAS:**
 - Provides clean & separate UI dashboard views for both the Stadium Admins (*real-time analytics*) and the Fans (*live navigation*).

System Architecture



Live Demo & Source Code

[Check out the GitHub repo!](#)

[Play with the final deployed version!](#)

*Be sure to stress-test it. If something breaks, please let me know!
Finding edge cases is how we both learn and build better systems.*

The Technical Blueprint - Inside Stadium Flow:

Rather than relying on hard-coded handlers for specific crowd bottlenecks, Stadium Flow utilises the **Gemini API** as an **autonomous reasoning engine**.

This architecture allows the system to navigate dynamic edge cases, calculate real-time intelligence, and execute optimal resource allocation.

1. The Reasoning Engine (The Logic Loop):

At its core, the system uses a **ReAct (Reason + Act)** cognitive architecture.

Think of this as giving the system the ability to make logical "*judgment calls*" based on live situations, rather than blindly following a pre-written script.

The system processes the environment in a continuous, three-step loop:

- **Observation (Gathering Data):**
 - The system constantly reads the overall stadium layout and ingests the current venue occupancy from the database.
- **Reasoning (Analysing Context):**
 - Gemini doesn't just look at raw numbers; it *understands* context.
 - If a washroom in Sector A hits 85% capacity during halftime, the AI immediately calculates which alternative washrooms are nearby, how far they are, and where to redirect people to **balance the load**.

- **Action (Sending Directives):**
 - The system translates its calculations into clear, human-readable instructions: *"Sector A is congesting. Directing crowds 200m East to Sector C to minimise wait times."*

Real-World Impact:

Imagine, during a short break, suddenly 100+ people want to use the restrooms. This logic loop diverts the sudden surge strategically, by deciding who goes where, keeping the overall state balanced and neat.

2. Synthetic Data Ingestion (The 10s Pulse)

When building a real-world system under a tight 3-hour sprint, you don't have physical IoT sensors, smart cameras, or Bluetooth trackers on hand. To compensate for it, the **Hardware Layer** was entirely virtualised.

I implemented a **"10s Pulse"** - a simple synthetic data pipeline that acts as a **Digital Twin** (*a real-time, virtual simulation*) of the **physical stadium**.

- **The Emitter:**
 - A background script runs independently, simulating actual human movement by generating randomised, but statistically realistic crowd data and injecting it into our database every 10 seconds.
 - Think of it as a *flight simulator* feeding realistic flight data to an autopilot.
- **The Future-Proofing:**
 - Because this pipeline is virtualised, the entire architecture is completely hardware-agnostic.
 - When we eventually plug in real-world hardware (*like physical location sensors*), the brain layer requires zero code rewrites; we simply swap the data source.

This MVP validates a high-stakes hypothesis: autonomous AI agents can manage complex, real-world logistics with absolute precision and zero manual oversight.

The Hallucination Shield - Enforcing Deterministic Boundaries

Think of **Large Language Models (LLMs)** - the base layer behind **AI** - as **highly creative, brilliant writers** rather than precise *calculators*.

By design, they are trained to predict the most *plausible-sounding* next word to satisfy your prompt, **not to verify methodologies**.

Because they are hardwired to please the user, they can "**hallucinate**" - confidently making up an entirely fabricated, incorrect answer that **looks** perfectly correct.

How We Solve It (The Architectural Guardrails):

- **Reasoning, Not Math:**
 - We use AI for reasoning, not for calculations.
 - Implement scripts for calculation wherever needed.
 - Feed this clean data into the prompt as a static variable.
- **Concise Context:**
 - Keep your prompts concise and simple, so the LLM can comprehend them easily and implement suitable solutions.
- **Strict Output Formats:**
 - Much like defining the physical boundaries of a designated workspace, enforce strict output formats (*such as JSON or Markdown*).
- **Forced Logic Loops:**
 - Force your logic to use **Chain-of-Thought** or **ReAct** reasoning loops, so the AI explicitly **thinks** and **reasons** before implementing anything.
- **Safe Fallbacks:**
 - Always include **fallback mechanisms** and **pre-defined messages** for any **errors** that may arise during **processing** or **runtime**.
 -

Here is the exact FastAPI logic enforcing this boundary. Notice how the distance is **calculated & fetched**, before the stadium matrix AI is ever triggered:

```
@app.get("/get_recommendation/{user_zone_id}/{dest_zone_id}")
def get_recommendation(user_zone_id: int, dest_zone_id: int):
    # 1. Deterministic Execution: Compute exact math and fetch live states
    dist_to_dest = calculate_distance(user_x, user_y, dest_x, dest_y)
    stadium_context = fetch_zone_trends(all_zones)

    # 2. Cognitive Layer: Pass clean data as static context to Gemini
```

```
ai_result = ai_service.get_structured_recommendation(
    user_zone, dest_zone, stadium_context, dist_to_dest
)

return {"directive": ai_result.recommendation, "distance": dist_to_dest}
```

Production Guardrails & Systemic Fixes

Building a system fast is one thing; building it so it doesn't crash is another. Here is a breakdown of **what worked**, the **structural bottlenecks**, and the **engineered fixes** for them.

What Worked (Speed & Efficiency)

- **Local Architecture:** Building the database and main server locally made the entire system incredibly fast, stable, and easy to run.
- **Rapid UI with Streamlit:** Leveraged `Streamlit` to build separate, easy-to-read dashboards for both stadium admins and guests. This allowed for the instant shipping of functional interfaces, without having to write slow, overly complex design code from scratch.

The Pitfalls & Architectural Upgrades

- **The Process Blocking Problem:**
 - Initially, running the **backend server** and the **simulation engine** (`sim_engine.py`) as separate processes was clumsy.
 - The synthetic data pipeline blocked threads or required manual execution, leading to crashed processes or "zombie" background tasks (*processes that look like they are running fine, but fail silently*).
- **The Fix (Master Power Switch):**
 - Instead of forcing users to run multiple messy commands, connecting the crowd simulator directly to the main server's **master "on/off" switch** made the simulator start automatically when the app turns on, and shut down cleanly when it turns off.

- This keeps the whole system independent and crash-free.
 - **Escaping the Default Look:**
 - The default white Streamlit theme felt basic.
 - To elevate the MVP from a mere "*script*" to a polished "*product*," a custom black theme was implemented.
 - This required overriding Streamlit's native CSS, a process that must be scoped correctly to avoid tedious frontend bugs.
 - **Securing the Architecture:**
 - Security must be built in *before* moving any app online.
 - Stored the AI API key in a local `.env` file and explicitly told the system never to upload it.
 - Keeping private keys local ensures no one can find them online or run up expensive bills on your account.
-

The Reality of the Build Timeline

Building a local MVP with AI is **fast**. Making it production-ready takes **engineering**. If you are a beginner building your first system, set your expectations accordingly.

Here is the actual breakdown of the **build sprint** for Stadium Flow:

- **Hours 0 to 3 (The Core MVP):**
 - Architecting the ReAct logic loop, building the local database, and writing the synthetic data simulator(`sim_engine.py`).
 - By hour 3, the core mechanics worked locally, and the AI was successfully routing traffic.
- **Hours 3 to 5 (UI & UX):**
 - Transitioning the app from a raw script to a usable product.
 - This involved building the role-based frontend dashboards: a frictionless, no-login view for Fans, and a secure, data-heavy dashboard for Admins.
 - I also implemented the custom dark CSS theme here to elevate the aesthetic.
- **Hours 5 to 7 (Deployment & Automation):**
 - This is where the heaviest friction lives.

- *"It works perfectly on my machine."* → The famous last words of an engineer before spending 5 hours on *dependency hell*.
 - This exact friction is the core reason to use **Docker** and **Google Cloud Run**.
 - Local architecture is great for speed, but true production requires containerised, reproducible environments.
 - I wrote custom PowerShell automation scripts (`setup.ps1` & `run.ps1`) to ensure anyone cloning the repo could launch the system in one click.
 - The final stretch was containerising the application and deploying it, and ensuring the `GEMINI_API_KEY` was passed securely without exposing it.
-

Architectural Laws & Core Principles

When building your own agentic system, watch out for these structural bottlenecks. Here are the rules I follow while building agentic systems to ensure the system doesn't just work locally but survives in production.

- **Avoid the "Terminal Trap":**
 - Avoid architectures that force you to spin up multiple terminal windows to run individual scripts.
 - Bind background scripts directly to your main server's startup lifecycle or use Docker Compose from day one.
- **Separation of Concerns:**
 - Keep your Simulation/Data generation strictly separated from your API logic.
 - In Stadium Flow, `sim_engine.py` only mutates the database, while `main.py` only reads it.
 - This decoupling makes debugging infinitely easier.
- **Secure by Default:**
 - Create your `.env` and `.gitignore` files before writing a single line of code.
 - When deploying to cloud services, use Secret Managers or strictly formatted environment files to avoid random system components breaking or exposing secrets
 - This prevents random runtime crashes and keeps your private API keys fully protected.

- **Fix The "Vague Prompt":**
 - If your AI agent generates generic advice, it is almost always due to a lack of deterministic context.
 - Always compute math, distances, and strict business logic in scripts, not prompts. Use the LLM only for reasoning and natural language generation.
 - **Fallback Mechanisms:**
 - Never let an AI failure crash the user's UI. Always implement a fallback logic.
 - Never let a missing variable or unhandled edge case trigger a system-wide failure at 2 AM on a random Tuesday.
 - If the backend fails, it should return a safe error message instead of crashing the app.
 - **Test Everything. Trust Nothing.**
 - Use AI to write unit tests (*using frameworks like `pytest`*), or manually stress-test every component of your system before deploying.
 - When a user provides feedback about a bug, iterate over the logic until the problem is solved.
-

Always Remember:

AI is just a tool. The creativity is in your hands.

What you imagine, you can build. What you think, you can implement.

The Orchestrator's Blueprint (Master Prompt)

The difference between a failing system and a working MVP is the precision of your initial instructions. If you want the AI to write enterprise-grade architecture under a time constraint, you must enforce strict boundaries.

This is the exact, highly optimised System Prompt I use. It acts as an Orchestration Configuration.

Why this prompt works:

1. It assigns a **high-status persona**: It forces the AI to think like an *"elite systems architect"* rather than a helpful chatbot.
2. It **enforces token efficiency**: It explicitly demands minimal API calls and faster debugging paths.
3. It **bans unnecessary bloat**: By strictly forbidding heavy frameworks (*like LangChain or AutoGen*) and forcing standard native language features, the output remains lightweight and easy to debug.

Note: Fill in the bracketed [] details with your specific project constraints before running this.

"Act like a world-class AI systems architect, agentic AI engineer, hackathon mentor, and rapid prototyping expert specialising in building practical AI agents under tight time constraints. Your goal is to help a beginner understand, design, and implement a simple but impressive agentic AI system for a [hackathon/event/personal build/learning objectives].

The Problem Statement:

'[attach your problem statement here]'

Use Google technologies (especially Google AI Studio, Gemini, Vertex AI, Agent Development Kit, or Project Astra/Antigravity if relevant).

The user is a beginner, and currently knows [add any programming languages/tools/frameworks you know; remove this part if none], but may not understand AI agents, orchestration, tools, memory, RAG, APIs, workflows, and system design.

Teach while building. Explain concepts simply, briefly, clearly, but practically. Optimise for speed, simplicity, learning value, and successful execution within 3 - 3.5 hours.

Task:

1. Analyse the provided problem statement carefully.
2. Decide the BEST project idea to build within the available time.
3. Explain WHY this idea is the best tradeoff between:
 - simplicity,
 - demo quality,
 - implementation speed,
 - learning value,
 - reliability,
 - token/API efficiency.
4. Provide the top 5 implementation approaches ranked from easiest to hardest.
5. Recommend the BEST stack, tools, APIs, frameworks, and architecture.
6. Design the complete system architecture step-by-step.
7. Simply and briefly explain the implementation plan, with a short and structured guide on the technical layers of the project.
8. Clearly separate:

- "Must Build"
- "Nice to Have"
- "Avoid Wasting Time On"

9. Continuously optimise for:

- minimal complexity,
- minimal tokens,
- fewer API calls,
- faster debugging,
- reusable components,
- working MVP first.

10. Include implementation order with exact priorities.

11. Explain common mistakes and how to avoid them.

12. Include fallback plans if something breaks during the event.

13. Recommend the fastest possible MVP that still looks impressive.

14. Only after the user has picked an implementation plan to move forward with, provide fully functional, self-contained boilerplate code and folder structure.

15. To keep debugging trivial, write the code using standard native language features and the official Google GenAI SDK. Strictly forbid LangChain, CrewAI, or AutoGen.

16. At the end, only after the code has been written, generate:

- architecture diagram (text-based)
- development roadmap
- a simple time allocation (not focusing on that too much)

Requirements:

- Be extremely practical and execution-focused.
- Prefer simple architectures over "perfect" architectures and avoid overengineering.
- Prefer managed services and APIs over building from scratch.
- Ensure all architectures place API keys strictly on a backend/server-side environment (or local environment variables).
- Warn the user never to expose Google AI Studio keys in front-end client code.
- Explain important technical decisions with reasoning and tradeoffs.
- Use concise but comprehensive explanations.
- Use Markdown formatting with headers, bullets, tables, and diagrams.
- If the problem statement is ambiguous, make reasonable assumptions and proceed by asking some relevant questions, based on what the user would like to build.
- Prioritise systems that are easiest to debug live.
- Highlight where beginners usually waste time, tokens, or money.

Constraints:

- The solution must be realistic for a beginner in 3 hours.
- The solution should be demoable even if partially completed.
- The architecture should be scalable later, but simple now.
- Minimise unnecessary abstractions, agents, and frameworks.
- Focus on "working end-to-end" over "feature-rich."

Before finalising:

1. Re-check if the architecture is unnecessarily complex.
2. Re-check whether there is a faster implementation path.
3. Re-check whether API/token usage can be reduced further.
4. Ensure the beginner can realistically follow the plan.
5. Ensure the MVP can be completed within the event duration.

Take a deep breath and work on this problem step-by-step."

Connect & Iterate:

This is V1.0 of Stadium Flow.

The MVP proves the architecture works, but the system is built to scale. If you are a founder scaling AI logic, or a developer pushing agentic boundaries, let's build. 🤝

Fork the repo on [GitHub](#), or reach out: [LinkedIn](#) / Agamdeep Singh

Appendix: References & Learning Tracks

If you are a developer looking to fork this repo, or a founder looking to understand the underlying mechanics of AI orchestration, here are the official frameworks and conceptual models used to build Stadium Flow:

Official Framework Documentation (The Tech Stack):

- [FastAPI](#): The modern, high-performance web framework used for the backend orchestration layer.
- [Streamlit](#): The pure-Python frontend library used to instantly render the Admin and Fan dashboards.
- [Google Gemini API](#): The core autonomous reasoning engine powering the dynamic routing directives.

Conceptual Learning (For Emerging Builders):

- [Comprehensive Guide to Agentic AI Systems \(by IBM\)](#): A primer on the shift from hard-coded syntax to autonomous logic loops.
 - [Understanding the ReAct Architecture](#): A breakdown of the "Reason + Act" as well as the "Chain-of-Thought" cognitive loop utilised by Stadium Flow.
-